

Research activity log for the matched SHD feasibility shift

ar067 · 18 July 2026 · Draft

Checkpoint CP-001

This initial checkpoint preserves the user-visible session through the request for an activity log and the first acknowledgement. The immutable private source is stored outside the repository. Its SHA-256 prefix is `f85b936a4a2e`. The session identifier is `019f76ab-932b-72c2-b163-463ead65d8d5`; the checkpoint identifier is `CP-001`; and the checkpoint time is `2026-07-18 19:49:27 UTC`. This checkpoint has no predecessor. Private paths, environment context, SSH paths, and the live pod identifier are marked explicitly where redacted.

Scientific ledger at CP-001

1. The locked exp066 design remained unchanged: seed 42, matched SHD input and readout settings, COBA without voltage-gradient dampening, and PING with a dampening factor of 1000.
2. The local 128-sample, two-epoch smoke passed for both cells with finite, active dynamics and no skipped updates. The shared input-scale adjustment was not used.
3. A mistaken smoke-gate interpretation of the active-cell fraction as saturation was corrected without retraining or changing a scientific parameter. The implementation is commit `e42b264`; the timestamped smoke Record is commit `3c29fe9`.
4. The first RunPod dispatch created no pod and cost nothing because the dispatch host lacked its expected deployment public key. The abort is commit `c98967d` and is also logged on pull request 50.
5. The unchanged pilot then started on one RTX 5090 pod at 0.99 USD per hour. COBA crossed the registered 20% criterion provisionally by epoch 14 with finite diagnostics and no skipped updates. The full pilot was still running at this checkpoint.
6. COBA training completed, but its first replay used synthetic input because loading a configuration did not itself activate SHD dataset mode. PING training continued. The pending runner correction explicitly selects SHD during replay and reuses validated training weights on retry.

Pending work at CP-001

1. Complete PING training and allow the first billable pod to self-remove.
2. Commit and push the replay fix, then use the permitted second attempt only to capture missing matched SHD rasters from validated weights.
3. Collect and validate the pilot artifacts, calculate exact spend, render `numbers.json`, figures, reproducer, exp066 report, and final ar066 Digest.
4. Build, push, update pull request 50, and stop at the human review gate.

Privacy-sanitized visible transcript

Message 1: 2026-07-18 19:19:51.440 UTC | user

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

```
# AGENTS.md instructions for [REDACTED_PRIVATE_PATH]

<INSTRUCTIONS>
> **demolab lab** – before working here, run `demolab docs` and follow what
it prints
> (the agent manual + runbook menu; no venv yet? `uvx demolab-cli docs`). A
user message
> that is just a NAME in CAPS (`HELP`, `LINT`, `DOCTOR`, ...) is a command –
the manual explains.

# pinglab

Spiking E/I (PING) networks, trained with surrogate gradients and diagnosed
via
Δt-stability. The `tools/snn` engine emits data → `experiments/` runners
render figures →
`writings/*.typ` publish via Typst.

## This lab's rules

- **No RunPod fan-outs without explicit permission** – `--runpod --live` (and
any
  pod-creating call) spends real money; default local. Same for anything
  Modal-dispatching.
- **Don't write to GitHub issues/PRs without explicit permission** – with
explicit
  permission, creating/posting an issue is fine; reading is always fine.
  Don't open
  branches/PRs unless asked – "commit and push" means commit + push to the
  current branch.
- **Free reign on notebooks; editing the cli (`tools/snn`) needs explicit
permission.**
- NEVER add Co-Authored-By, "Generated with Claude Code", or any other AI-
attribution trailers to commit messages or PR descriptions. Commits are
  authored by Eoin alone.

## Session hygiene

Eoin tends to let sessions grow far too large. When the conversation
**changes topic** (a new
  experiment, a pivot from writing to debugging, a finished runbook, an
  unrelated question), the
  agent should proactively say so and suggest the right cleanup – don't wait to
  be asked:
```

- `**/clear**` when the new topic shares nothing with the old one – carry-over context is pure cost. Offer a one-paragraph handoff summary to paste into the fresh session if useful.
- `**/compact**` when the thread continues but the history is mostly spent (long tool output, finished sub-tasks, resolved debugging).

Suggest at natural boundaries (task done, topic pivot) – not mid-task, and at most once per boundary; if declined, drop it until the next one.

</INSTRUCTIONS>

[REDACTED_ENVIRONMENT_CONTEXT]

Message 2: 2026-07-18 19:19:51.482 UTC | user

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

/goal Execute the locked SHD feasibility mandate in writings/ar066.typ on the current autoresearch/shd branch.

Outcome:

Implement and complete exp066, establishing that matched Dale-constrained COBA and PING networks both train above chance on a small SHD task and produce directly comparable learning curves and input/E/I spike rasters.

Follow the staged protocol and scientific criteria in ar066 exactly:

- one seed;
- local 128-sample, 2-epoch smoke stage first;
- if both cells are finite and active, run the registered 1,000-sample, 20-epoch pilot on RunPod;
- COBA uses no voltage-gradient dampening; PING uses dampening 1000;
- input and readout settings remain matched;
- only the single pre-registered shared input-scale adjustment is allowed;
- do not change the hypothesis, baseline, success criterion, or kill criterion after seeing results.

Constraints:

- RunPod spending is explicitly authorized up to a hard total ceiling of \$40, with at most one pod at a time. Reap the pod when finished.
- Do not edit tools/snn without asking me first. Prefer implementing the experiment through experiments/exp066.py and existing CLI capabilities.
- Do not merge the PR or modify main.
- Do not run the full test suite on this 4 GB Hetzner host. Use focused lightweight checks and demolab build.
- Stop and ask if completion requires changing the registered scientific design, editing tools/snn, exceeding the compute budget, or taking another

action requiring authority.

Logging and deliverables:

- Record implementation and debugging work in focused commits with clear messages.
- Record scientific events, results, failures, anomalies, and conclusions in the Record and Digest of writings/ar066.typ. Link relevant commits when implementation affects interpretation.
- Produce artifacts/data/exp066 with provenance, numbers.json, reproducer, training curves, firing-rate diagnostics, and matched rasters.
- Create writings/exp066.typ as a cold-readable experiment report whose numbers and figures come from the recorded artifacts.
- Build successfully, commit each meaningful attempt including killed attempts, push autoresearch/shd, and update PR #50.
- Finish at the human review gate with a concise evidence summary, exact compute spend, validation performed, and links to the rendered ar066/exp066 files and PR. Do not merge.

Message 3: 2026-07-18 19:19:55.040 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

I'll execute the registered stages in order, keep the scientific design locked, and stop at the human review gate. I'll begin by loading the lab runbook and the GitHub workflow instructions, then inspect the mandate and current branch/state before touching files or compute.

Message 4: 2026-07-18 19:20:01.168 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

`demolab` is not installed on PATH, so I'm using the repository-prescribed `uvx demolab-cli docs` fallback. Because the task explicitly includes updating PR #50, I'm also applying the GitHub orientation skill before any GitHub interaction.

Message 5: 2026-07-18 19:20:09.067 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The lab manual routes this request through the `AUTORESEARCH` runbook. The GitHub workflow also routes commit/push work to its publish skill; I'm loading both in full now, along with the repository state and the locked mandate, before starting the smoke stage.

Message 6: 2026-07-18 19:20:21.955 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The runbook resolves this as an already-running night: the locked mandate and `autoresearch/shd` branch already exist, so I'll resume in place rather than allocate anything new. The registered ceiling is one local smoke stage plus at most two RunPod attempts per model, two hours each, eight hours for the shift, \$40 total, and one pod at a time. Next I'm checking the full scientific contract, house style, PR/auth state, and existing SHD/RunPod interfaces.

Message 7: 2026-07-18 19:21:02.399 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

No pod is currently running, the worktree is clean, PR #50 is the existing draft for this branch, and GitHub authentication is valid. The old SHD runner is scientifically incompatible (free signed recurrence), so I'm using it only for artifact conventions; the new runner will invoke the existing CLI with the exact registered Dale-constrained settings and no `tools/snn` edits.

Message 8: 2026-07-18 19:21:28.913 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

I've fixed the implementation shape: default execution performs only the registered local smoke; RunPod mode packs both pilot cells sequentially onto one pod; each pilot cell also records the same preselected test indices (0, 1, 2) before any prediction-based selection. Collection and publication happen only after both validated cells return. This keeps the one-pod limit and makes the smoke-to-cloud commit boundary auditable.

Message 9: 2026-07-18 19:23:17.846 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The runner compiles and exposes only lifecycle/dispatch flags; all scientific parameters are hardcoded. I'm starting the 128-sample, 2-epoch local smoke now. It will train COBA first, then PING, and will abort before any cloud action if either cell is non-finite, silent, saturated, misconfigured, or has skipped updates.

Message 10: 2026-07-18 19:24:23.402 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

COBA has loaded correctly with the registered configuration. Its initial excitatory activity is finite (about 19 Hz); the displayed 100% "act" means every excitatory cell fired at least once during the observation window, not timestep-wise saturation. The smoke is still compiling/training, so I'm waiting for the epoch metrics before applying the registered gate.

Message 11: 2026-07-18 19:28:09.388 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The smoke data are finite and active with no skipped updates: COBA ends at 20.5 Hz E activity and 10.2% accuracy; PING at 2.53 Hz E, 13.0 Hz I, and 6.25% accuracy. My runner incorrectly treated “100% of cells fired at least once” as saturation, even though COBA’s 20.5 Hz rate is far below the 1,000 Hz timestep ceiling. That is an implementation bug in the gate, not a scientific failure or grounds for the shared adjustment. I’m correcting the gate to use the engine’s actual saturation definition, then will revalidate the already-produced smoke artifacts without retraining.

Message 12: 2026-07-18 19:28:41.947 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The corrected gate passes both smoke cells, so the pre-registered shared input-scale adjustment is not used. I’m now validating the RunPod plan in dry-run mode, then committing the runner as the first focused implementation attempt. After that I’ll add the smoke event to the scientific Record with the implementation commit linked, build, and push the exact commit that the pod will check out.

Message 13: 2026-07-18 19:29:50.707 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The build itself passed, but this toolchain version regenerated every historical PDF and resurrected six stale PDFs from removed entries. Those are build byproducts from this run, not user work. I’m removing only those exact stale outputs and restoring only the historical PDFs regenerated by my build, while retaining `ar066.pdf` and `book.pdf`. Then I’ll commit the focused Record update and push.

Message 14: 2026-07-18 19:30:12.469 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The passing smoke is now committed and pushed at `3c29fe9`; the runner dry-run confirms exactly one 5090 pod with the two cells executed sequentially. I’m crossing the authorized compute gate now and creating that single pod. The pod has a hard two-hour self-removal backstop, so the maximum exposure for this attempt is \$2.18 at the configured \$1.09/hour ceiling, well below the \$40 mandate ceiling.

Message 15: 2026-07-18 19:31:26.753 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The first cloud dispatch attempt created zero pods and spent \$0.00: provisioning failed before the API call because this host lacks the helper's expected `[REDACTED\_SSH\_PATH]`. No scientific cell ran. I'm checking for an existing public key or a credential-free supported path; if none exists, I can generate a local deployment key without changing `tools/snn` or the registered design, then record this failed engineering attempt in its own commit as required.

Message 16: 2026-07-18 19:32:27.385 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The zero-pod abort is now preserved in `c98967d` and the host has a dedicated local deployment key. I'm retrying the same unchanged pilot plan; this remains the first billable attempt because the prior dispatch never created a pod.

Message 17: 2026-07-18 19:32:54.110 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The pod `[REDACTED\_POD\_ID]` is running at the actual rate of \$0.99/hour. Both registered cells are assigned sequentially, so concurrency remains one. I'll monitor the pod and shared artifacts; it should self-remove after completing COBA, PING, and the six fixed-position raster captures.

Message 18: 2026-07-18 19:34:35.794 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The pod is still in RunPod's startup/image-pull phase and has not exposed SSH yet. Billing is accruing at \$0.99/hour, currently well under \$0.03. This is within the normal cold-start range documented by the lab helper, so I'm continuing to monitor rather than creating any second pod.

Message 19: 2026-07-18 19:36:57.974 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

Cold start is at roughly three minutes and still reports "pod not ready"; no duplicate action is being taken. The helper's documented authenticated-pull range is about two to four minutes, with longer outliers, and the two-hour kill switch remains armed. Current accrued cost is about \$0.05.

Message 20: 2026-07-18 19:38:15.498 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

The pod is ready and COBA is actively training on the GPU; the shared volume already contains its config and live metrics stream. I confirmed the process directly with the deployment key. I'll monitor epoch progress and intervene only if the run exceeds its two-hour backstop or emits non-finite/skipped-update diagnostics.

Message 21: 2026-07-18 19:39:36.756 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

COBA is progressing normally: two epochs are complete, metrics are finite, and the pilot is running at roughly one epoch every 20–25 seconds after compilation. At that rate both cells plus rasters should finish comfortably within the backstop; accrued cost is still under \$0.10.

Message 22: 2026-07-18 19:41:24.042 UTC | user

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

also make logs by commenting on the pr when needed, and in the logs in the demolab article make sure to timestamp them

Message 23: 2026-07-18 19:41:32.351 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

Understood. I'll timestamp every Record entry in `ar066.typ` and use PR #50 comments for engineering events a reviewer should see, including the zero-pod dispatch abort and any later cloud anomalies. I'll keep the scientific Record focused on scientific events while the PR thread carries operational detail. COBA is currently at epoch 7/20 and progressing normally.

Message 24: 2026-07-18 19:44:24.284 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

At COBA epoch 14, the registered accuracy criterion is already crossed (28.1% test accuracy), with finite loss, 24.1 Hz test E rate, zero skipped steps, and zero non-finite forward batches. This is provisional until the full 20 epochs and PING cell complete; I'm not changing or stopping the registered run early. The PR now contains the timestamped infrastructure log.

Message 25: 2026-07-18 19:49:01.318 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

I found an engineering anomaly in the evidence capture: COBA training completed, but the existing CLI replay defaulted to synthetic input because `--load-config` did not activate dataset mode by itself, so no SHD snapshot was emitted. PING training is continuing unchanged; its scientific training data are valid. I'm fixing only the runner's replay command by explicitly selecting SHD and making retries reuse validated training weights, so the permitted second attempt captures missing rasters without retraining valid cells. No registered setting or `tools/snn` code changes.

Message 26: 2026-07-18 19:49:09.189 UTC | user

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

Create another research activity-log Demolab article now and make an immediate checkpoint commit containing the thread so far. Preserve every user-visible message from this Codex session verbatim and chronologically, including user messages and your commentary/final messages. Record each message's timestamp, role, session ID, and checkpoint ID. Do not replace visible messages with summaries. Exclude hidden chain-of-thought and internal reasoning. Also record the scientific decisions, actions, evidence, failures, costs, commits, results, and pending work represented in the transcript so far. Before writing anything publishable, sanitize credentials, tokens, SSH material, IP addresses, environment values, private paths, and sensitive infrastructure details. Fail closed if sanitization is uncertain. Preserve a private immutable copy of the raw transcript and include its SHA-256 prefix hash in the checkpoint metadata. Build and validate the Demolab article, then commit this initial checkpoint immediately and push it to the existing programme branch so the current PR's GitHub Pages preview updates. Do not create a new branch or PR. Afterward, continue updating the same article at meaningful scientific milestones using timestamped, hash-linked checkpoints, then continue with the scientific goal

Message 27: 2026-07-18 19:49:27.345 UTC | assistant

Session: 019f76ab-932b-72c2-b163-463ead65d8d5. Checkpoint: CP-001.

I'll add a separate timestamped activity-log article on the existing branch and checkpoint it before continuing the experiment. Privacy takes precedence over verbatim publication where the transcript contains infrastructure identifiers: the private raw copy will remain outside the repository with a SHA-256 prefix recorded, while the publishable transcript will preserve message text and order but redact only sensitive substrings. I'll fail closed if I cannot identify the authoritative session transcript or sanitize it confidently. The running pod remains under its existing self-removal backstop while I checkpoint.